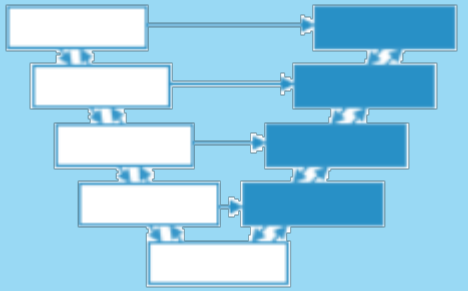
Collaboration, Quality, Test

Prof. Dr. Jóakim von Kistowski

Check-In

Recap:
What do we need to consider when testing in **iterative and incremental software development models**?

V-Model – Test levels (1/2)



Who's who?

Acceptance Test

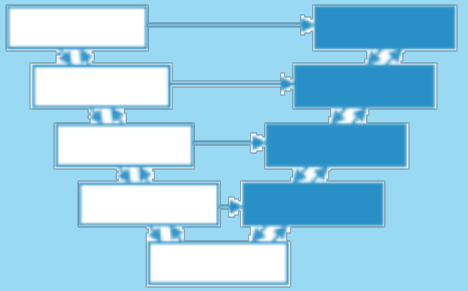
System Test

Integration Test

Component Test

- "I check whether the system as a whole fulfills the specified requirements."
- "I am there to check whether each software or hardware component fulfills its specification."
- "I check whether the components interact as defined in the technical system design."
- "I check whether the system has the agreed performance characteristics from the customer's point of view."

V-Model – Test levels (2/2)

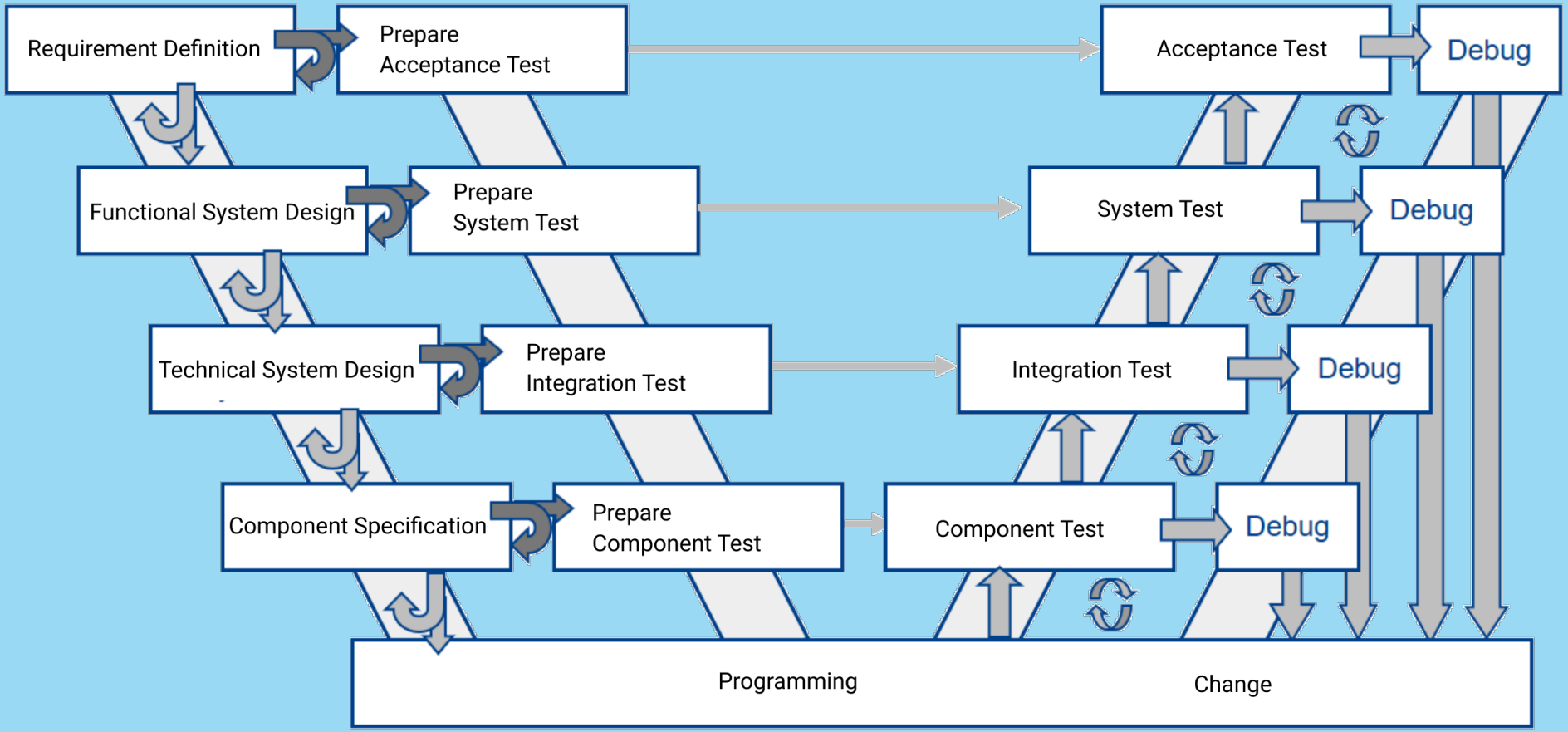


- **Component test**: Does every software or hardware component fulfill its specification?
- **(Component) Integration test***: Do components interact as specified in the technical system design?
- **System test**: Does the system as a whole fulfill the specified requirements?
- **Acceptance test**: Does the system exhibit agreed performance characteristics from the customer's point of view? inding errors is easiest on the same abstraction level

Additional type of integration test:

- **System integration test**: after system test, tests the interaction of different systems or between hardware and software
- **Unit Integration Test** = Component Integration Test

W-Modell – What is the reasoning behind it?



after Spillner/Roßner/Winter/Linz
Praxiswissen Softwaretest - Testmanagement
dpunkt, 2014, Chapter 3.3.2

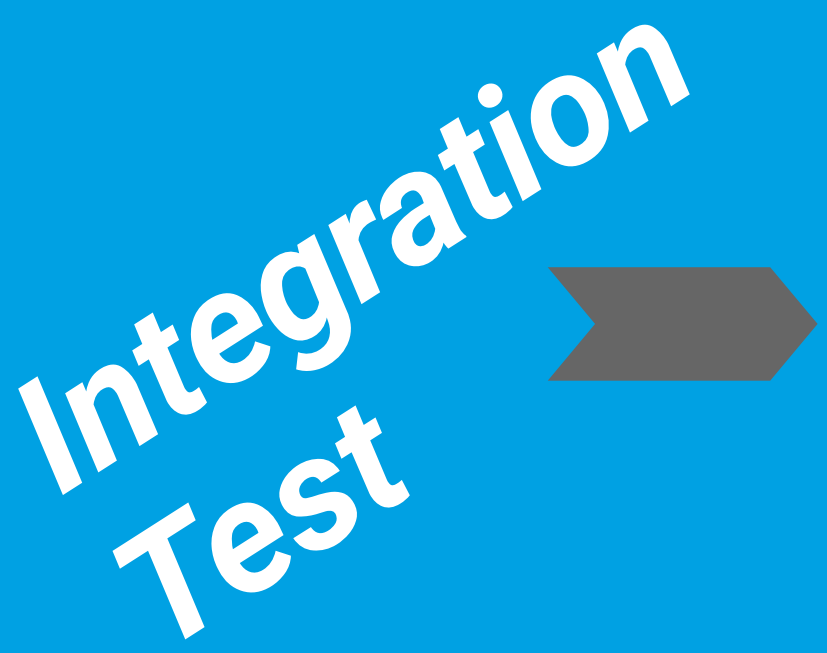
Review, Previews, Documents

Test cases, Test environments

test, debug, change, re-test

TH Aschaffenburg
university of applied sciences

Chapter 2: Testing Throughout the Software Development Lifecycle

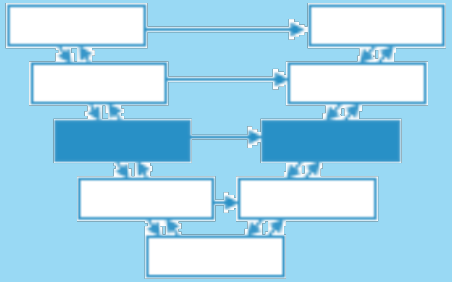


Testing in the Context of a Software Development Lifecycle

Test Levels and Test Types

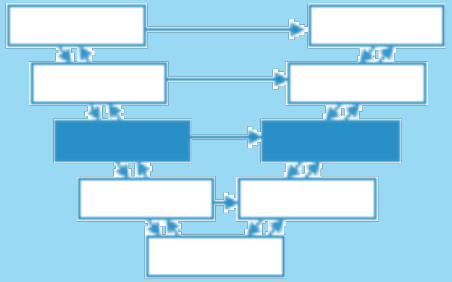
Maintenance Testing

Integration Test – Terms



- Integration test: second test stage after component testing
- Prerequisites
  - Component tests have concluded
  - Uncovered defects have been fixed
- Integration
  - Combine several components into larger subsystems
  - **Responsible**: developers, testers or **special integration teams**
- Integration test
  - Test whether the individual components work together
- Goals
  - Find interface failures
  - Finding failures in component interaction

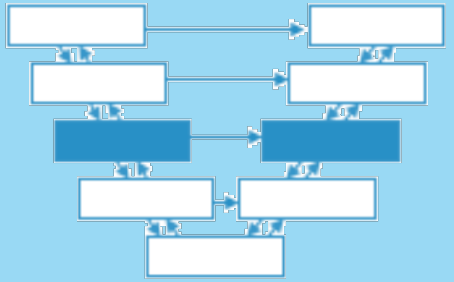
Integration Test – Test Basis



- Software and system design
- Sequence diagrams
- Specifications of interfaces and communication protocols
- Use cases
- Architecture at component or system level
- Workflows
- External interface definitions (API)

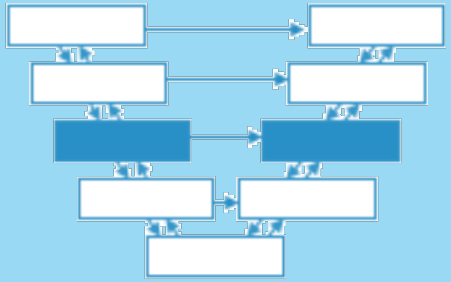
TH Aschaffenburg
university of applied sciences

Integration Test – Test Objects



- Integrate & test individual modules step by step to form larger units
  - subsystems
  - databases
  - infrastructure
  - interfaces
  - APIs
  - Microservices
- Each subsystem can be the basis for the integration of larger units

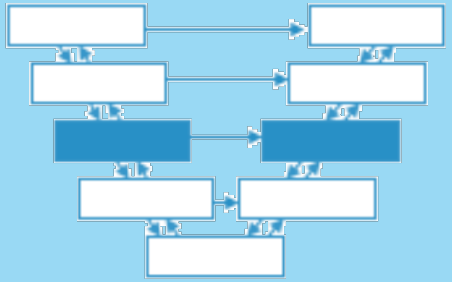
Integration Test – System Integration Test



Several potential integration levels:

- **Component integration test**
  - after component test
  - Focus: **Interaction of components**
  - Usually part of continuous integration
  - Often responsibility of the developers
- System integration test
  - After system test (also possible in parallel to system test)
  - Focus: **interaction of systems** (incl. hardware), packages, microservices
  - Interactions and interfaces of third parties can also be covered
  - **Often responsibility of the testers**

Integration Test – Defects and Failures



- Typical defects / failures:
  - Component transmits **incorrect data**, receiving component crashes (functional error of a component, incompatible interface formats, protocol error)
  - Receiving components interpret data incorrectly (functional error, contradictory or misinterpreted specifications)
  - Data is transferred correctly, but at the wrong time (timing problem) or at high frequency (throughput or load problem).
- None of these failures can be detected in the component test
- Failure only through interaction of components

Question

Why don’t we just skip component testing and do everything via integration test?

(this actually happens a lot in practice …)

What are the drawbacks of doing away with integration testing?

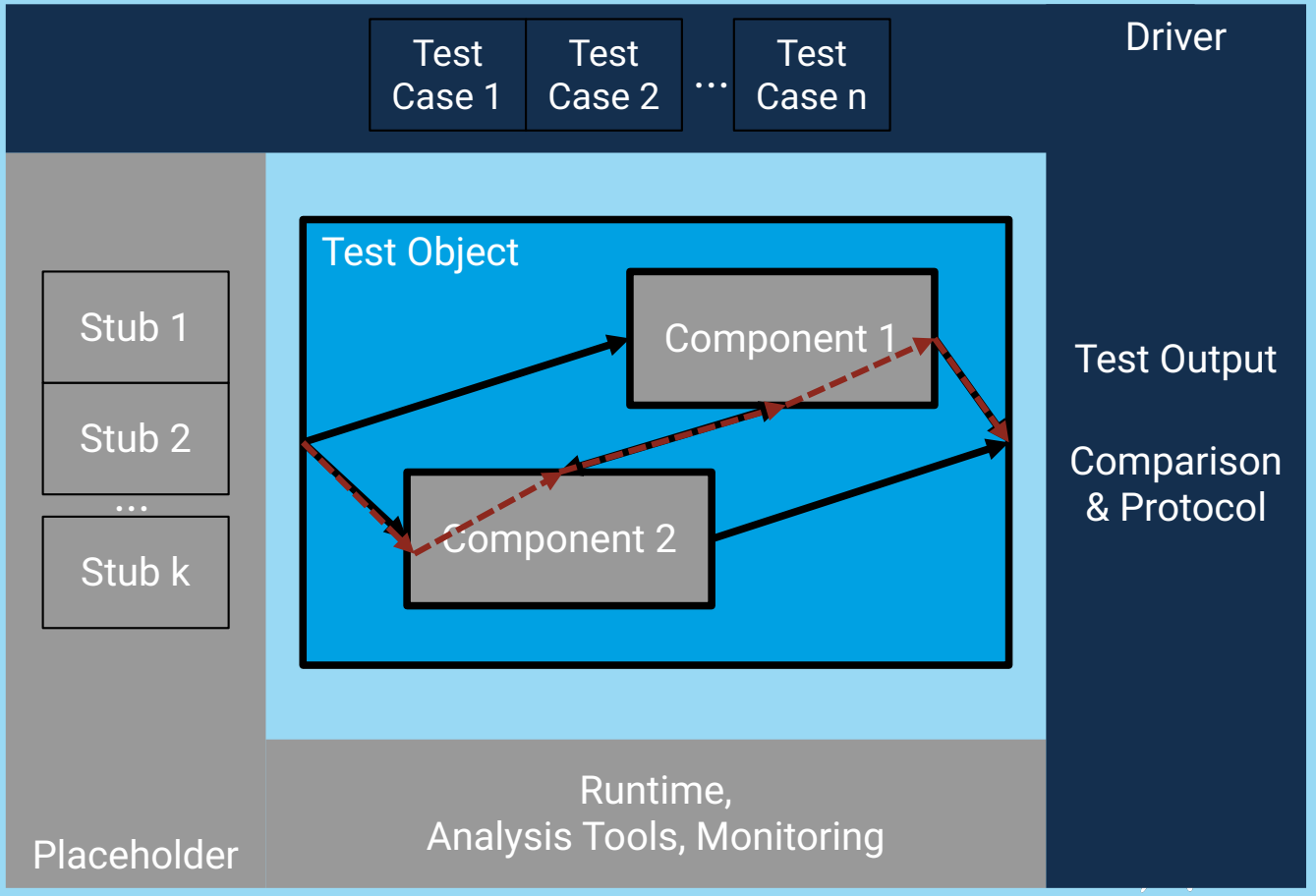
The greater the scope of an integration, the …

more difficult the isolation of defects

greater the time required to fix errors

Integration Test – Test Environment

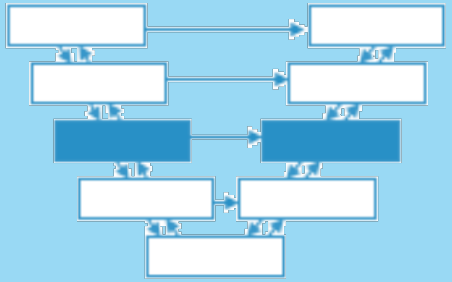
- Integration test with drivers: import test data, log results
- Reuse of existing component test drivers
- Additional diagnostic tool (monitors) for interface monitoring



Question

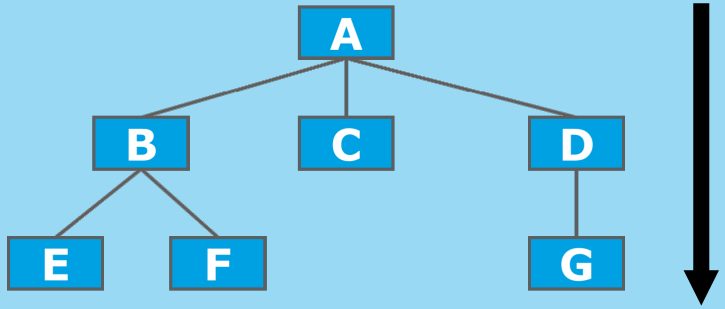
In what order do we integrate components?

Integration Test – Integration Test Strategy (1/5)



Top-down integration

- Test starts with component that calls others but is not called itself
- Successive integration of components in lower system layers
- Subordinate components replaced by placeholders
- Higher layer components under test used as driver



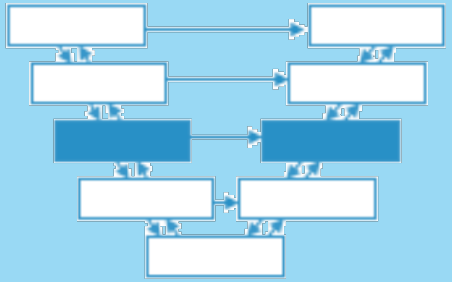
Advantage

- no / simple drivers required, as they consist of already tested components

Disadvantage

- subordinate, not yet integrated components must be replaced by placeholders

Integration Test – Integration Test Strategy (2/5)



Bottom-up integration

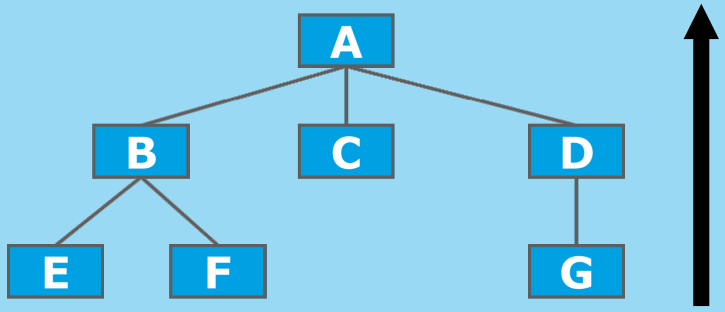
- Test starts with basic components that do not call others
- Successive integration of components with subsequent testing

Advantage

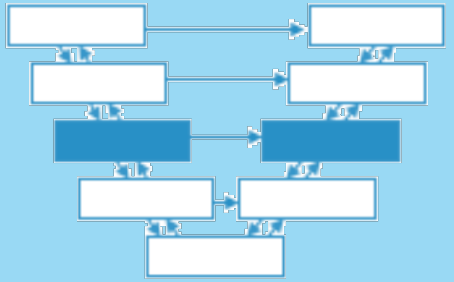
- No placeholders required.

Disadvantage

- Superordinate components must be simulated by drivers

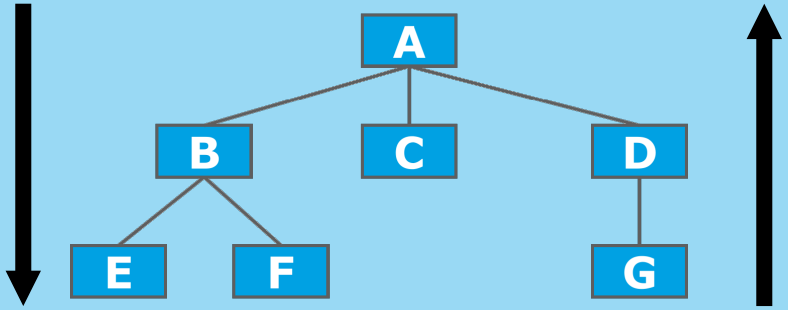


Integration Test – Integration Test Strategy (3/5)

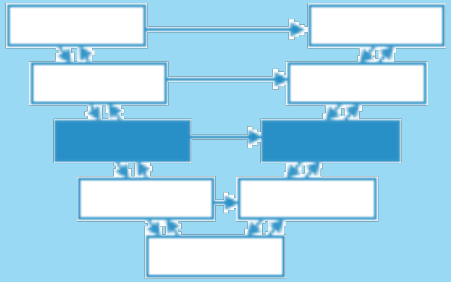


Comments

- Top-down or bottom-up approach can only be used for hierarchically structured systems (not super common)
- In practice, a mix of both integration strategies is commonly used



Integration Test – Integration Test Strategy (4/5)



Ad-hoc Integration

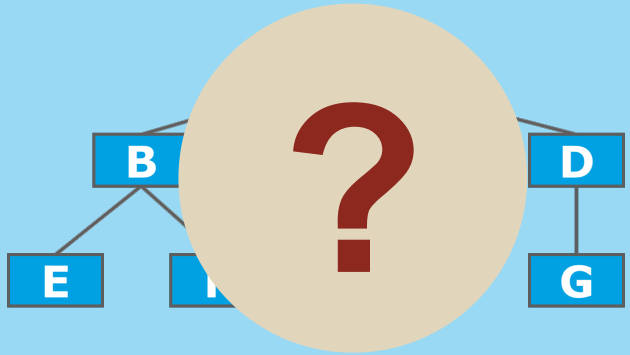
- Integrate components in (random) order of completion
- Check feasibility of integration directly after component test

Advantage

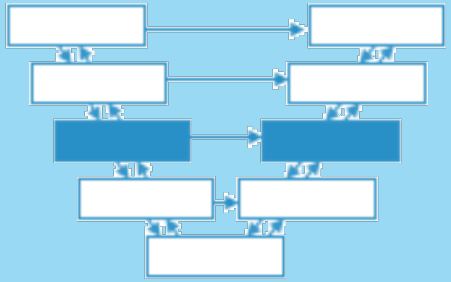
- Time saving, as earliest possible integration

Disadvantage

- Placeholders and drivers required

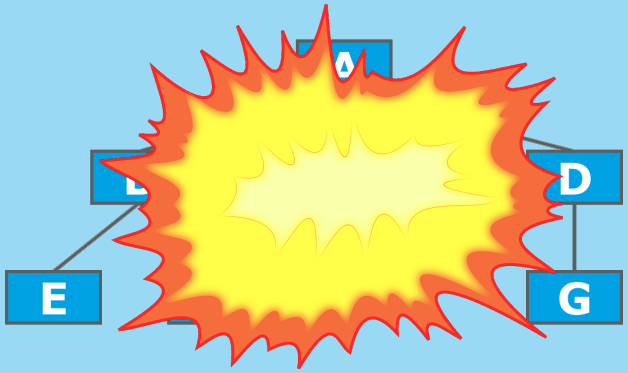


Integration Test – Integration Test Strategy (5/5)



Non-incremental Integration – *big-bang* Integration

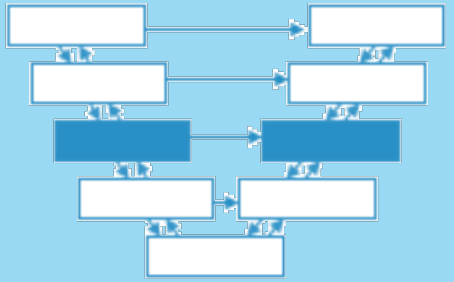
- Wait until all software components have been developed
- Integrate everything at once. At worst, without component tests
- Extreme version: integrate software and hardware elements at once

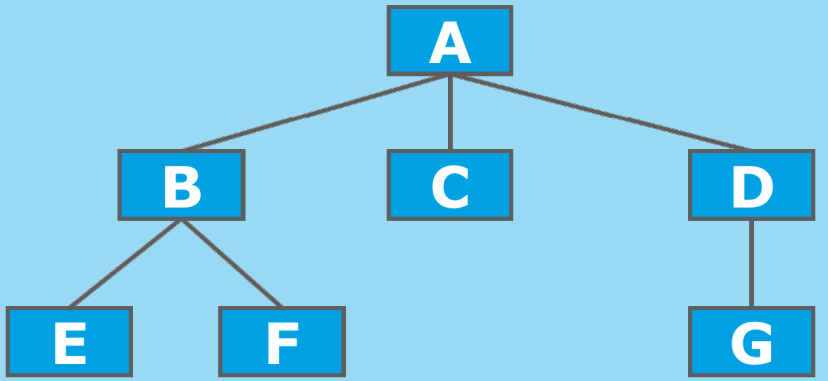


Disadvantages

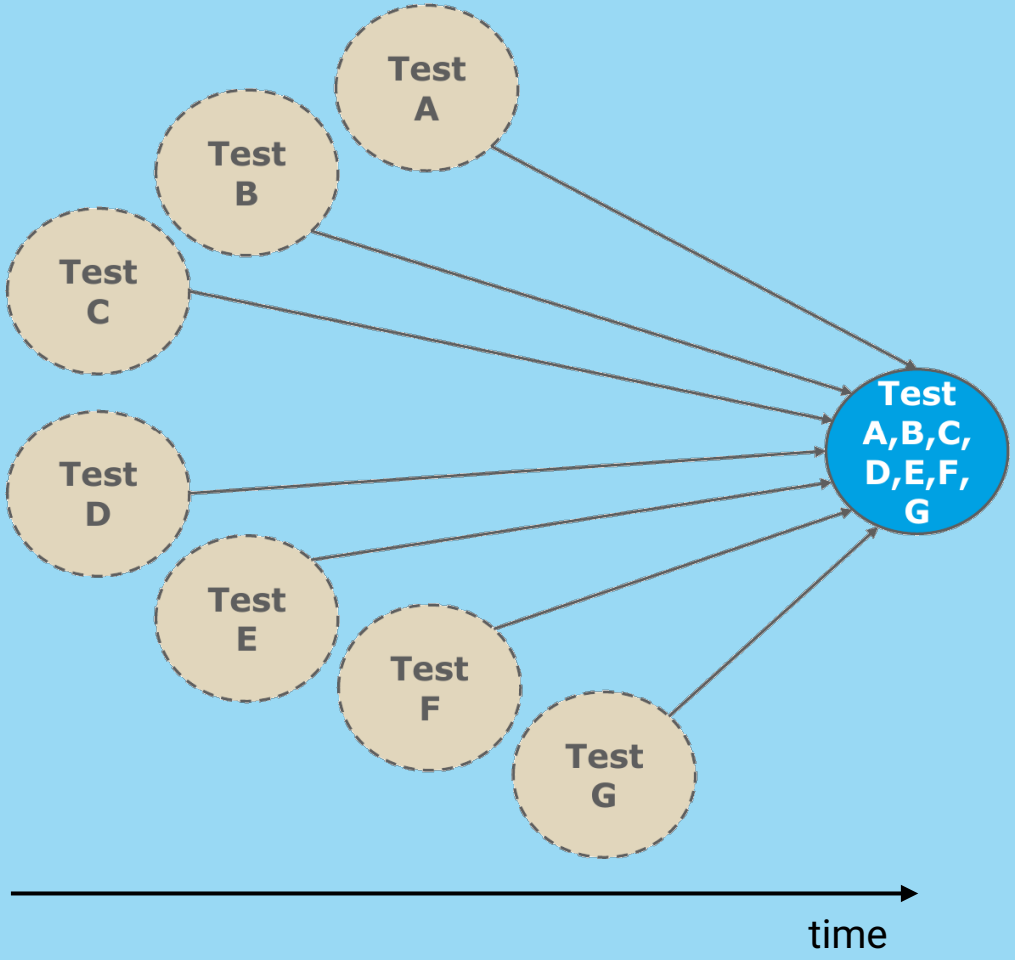
- Waiting time until the *big bang* is lost testing time
- Failures occur all at once
- Very difficult to ensure that the system works at all
- Finding and correcting defects is extremely difficult

Integration Test – Integration Test Strategy with Example (1/4)

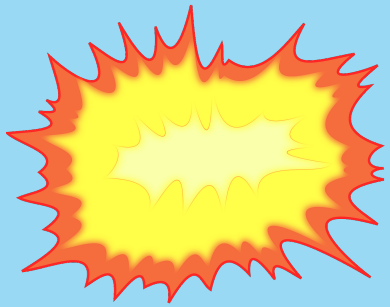


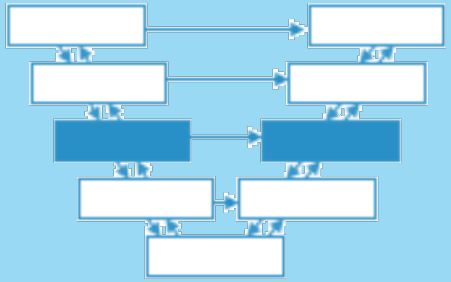


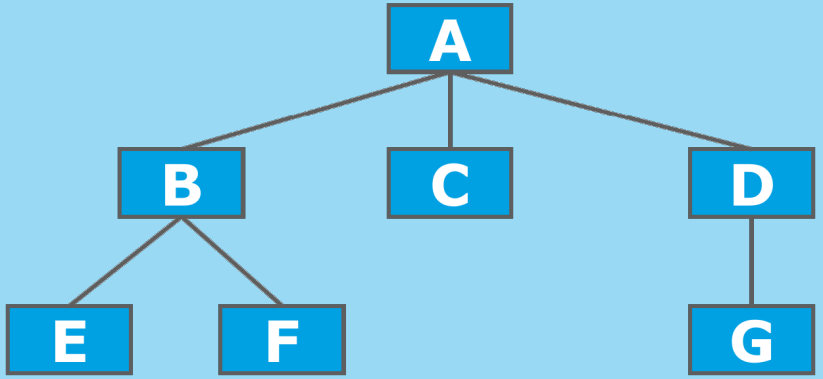
Integration Test – Integration Test Strategy with Example (2/4)



***big-bang* Integration**





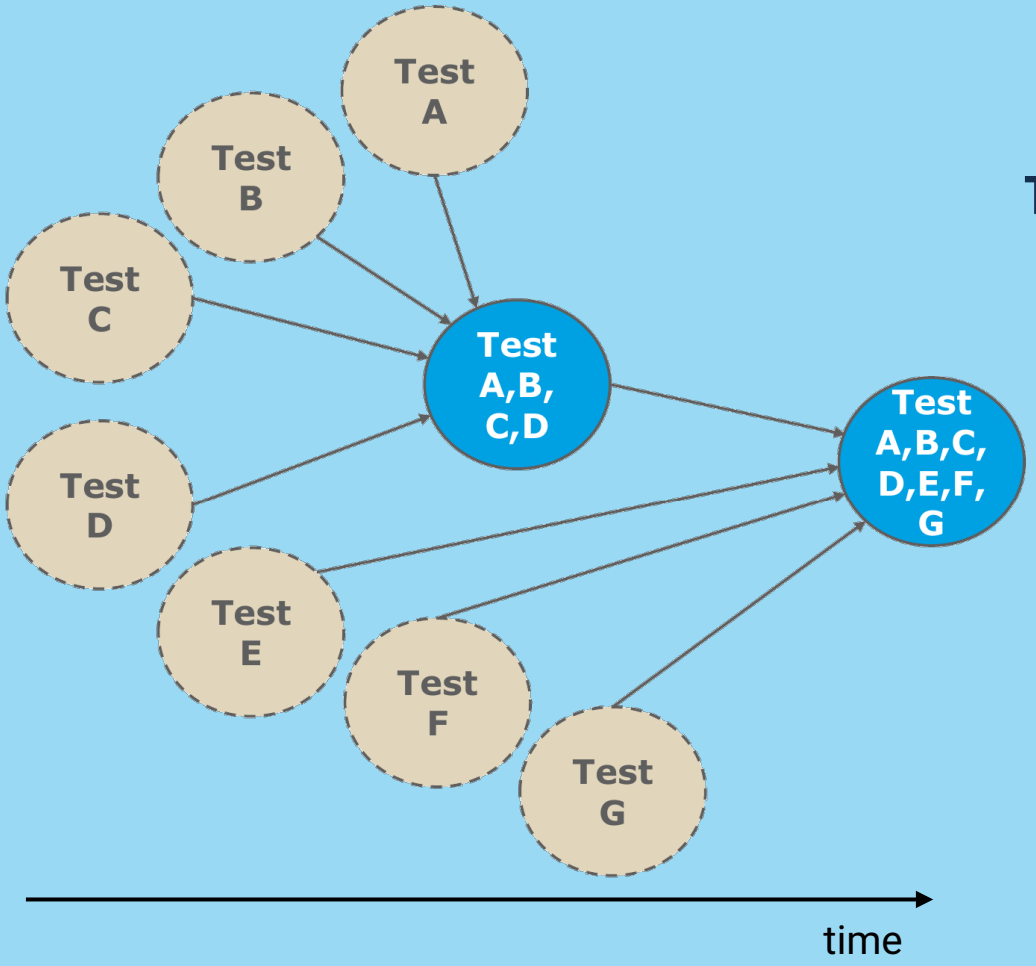


Component Test

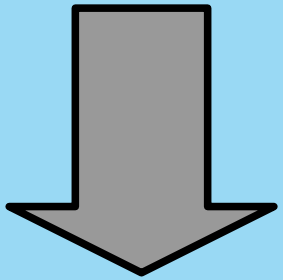
Integration Test

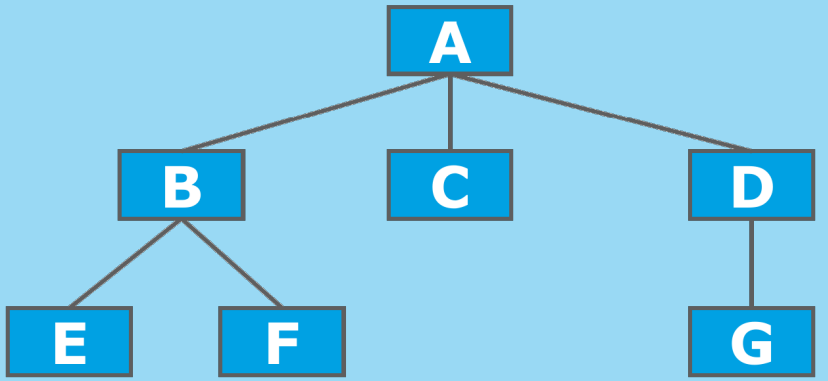
TH Aschaffenburg
university of applied sciences

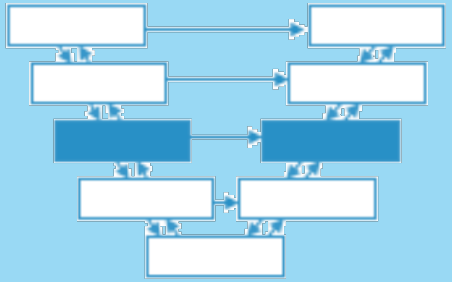
Integration Test – Integration Test Strategy with Example (3/4)



Top-Down Integration





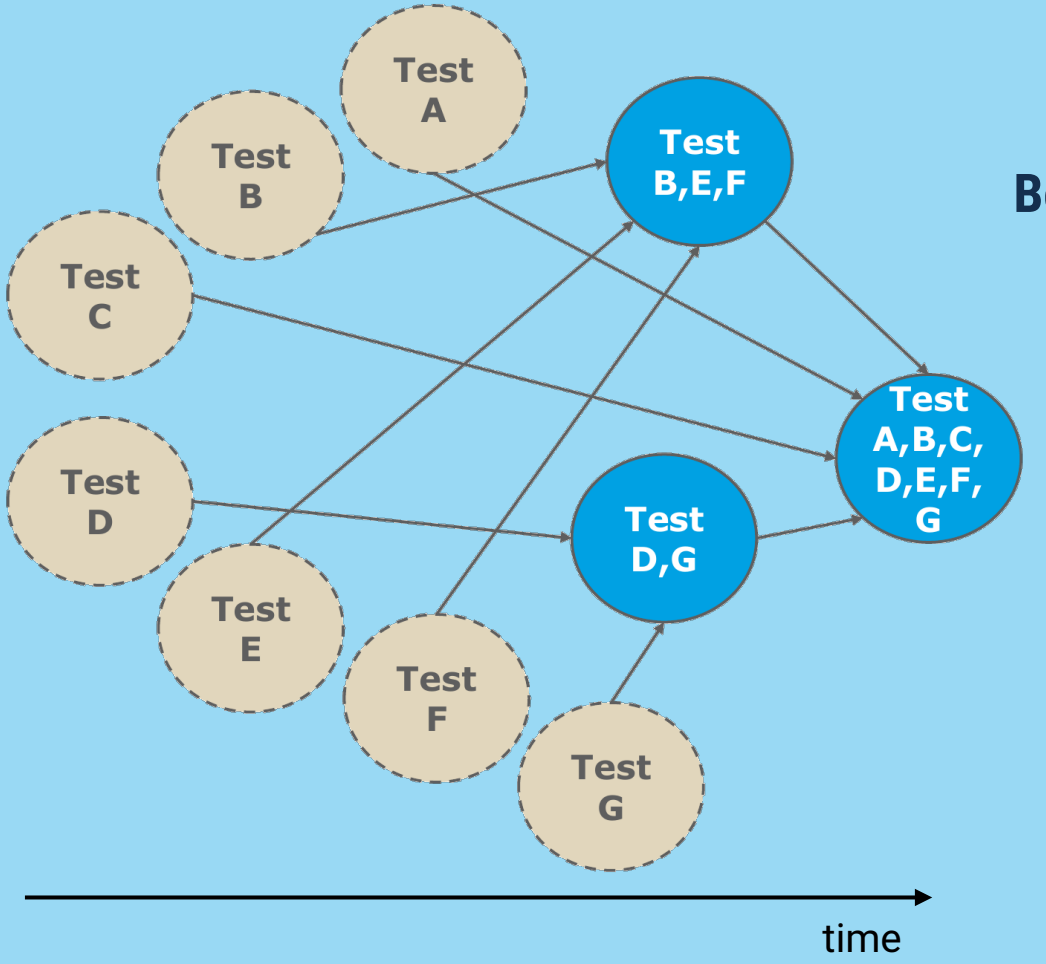


Component Test

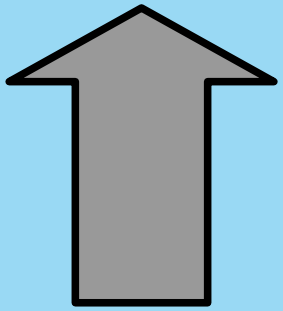
Integration Test

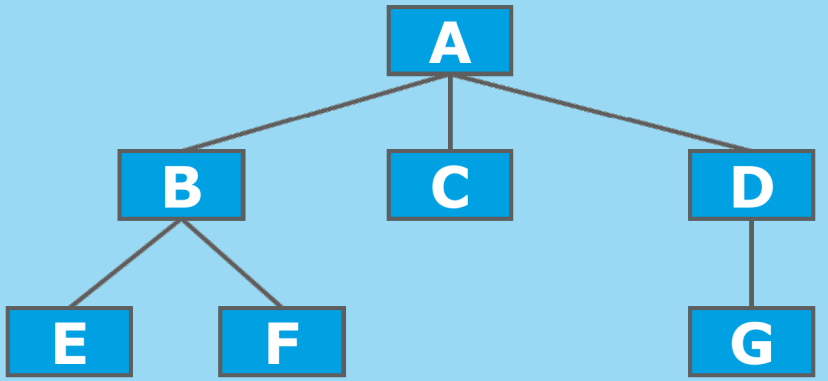
TH Aschaffenburg
university of applied sciences

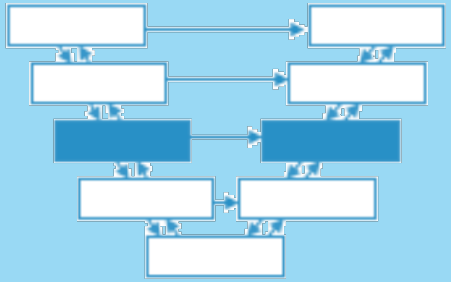
Integration Test – Integration Test Strategy with Example (4/4)



Bottom-Up Integration





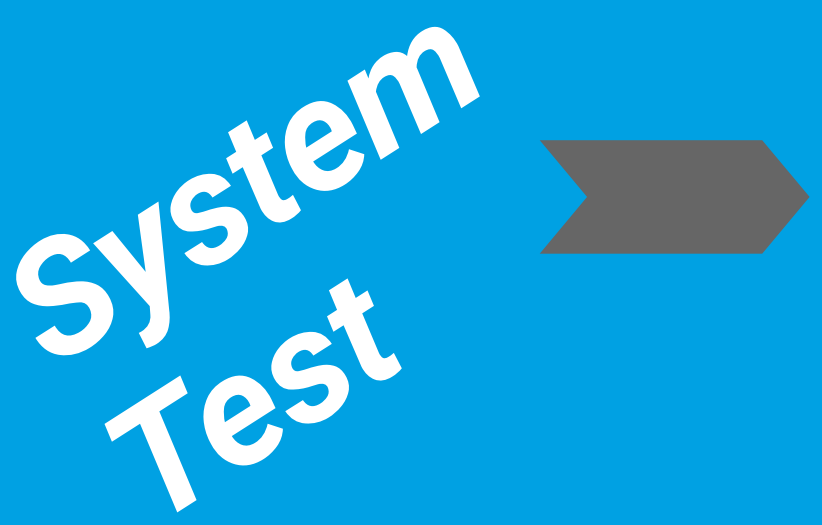


Component Test

Integration Test

TH Aschaffenburg
university of applied sciences

Chapter 2: Testing Throughout the Software Development Lifecycle



Testing in the Context of a Software Development Lifecycle

Test Levels and Test Types

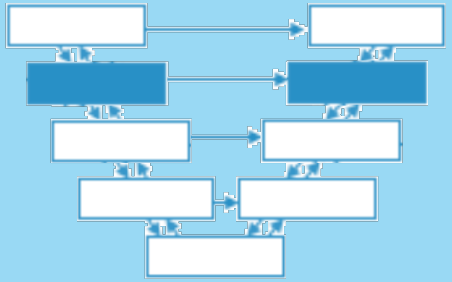
Maintenance Testing

Question

Now that we have tested all components by themselves and integrated…

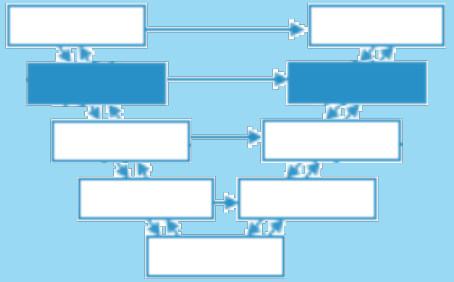
Why do we need to run a system test?

System Test – Terms



- System test: after the completion of integration test
- Focus: does the product fulfill the specified requirements?
- Motivation
  - Previous test stages: Testing from the perspective of the software manufacturer
  - System test: Testing from the perspective of customers and users
  - Requirements fully and appropriately implemented?
  - System functions and properties can often only be tested after integration of **all (!) system** components

System Test – Test Objectives



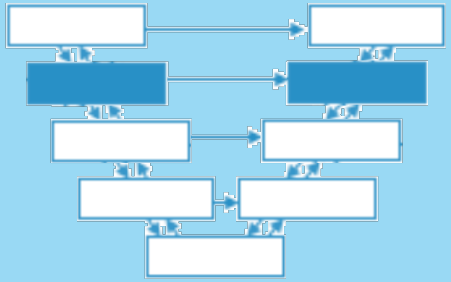
- **Consideration of system as a whole (functional and non-functional)**
- Risk reduction, finding of failures (and defects)
- On failure: Do not pass on to higher test levels / production
- Partially: Verification of data quality
- Create confidence in the quality of the system as a whole
- Collect information for release decision

Question

What can be used as a test basis for the system test?

System Test – Test Basis

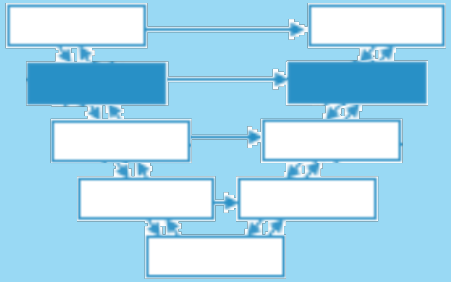
Examples for test basis in system testing



- (non-)functional requirement specifications
- Use cases, epics and user stories
- Risk analysis reports
- System behavior Models
- Documentation and user guides

System Test – Test Objects

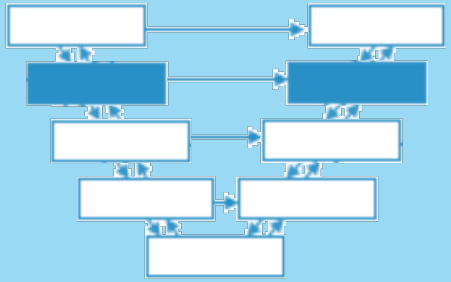
Common test objects in system testing



- Systems under test (SUT)
- Applications
- Hardware/software systems
- Operating systems
- System configuration and configuration data

System Test – Common Defects and Failures

Common failures and defects in system testing

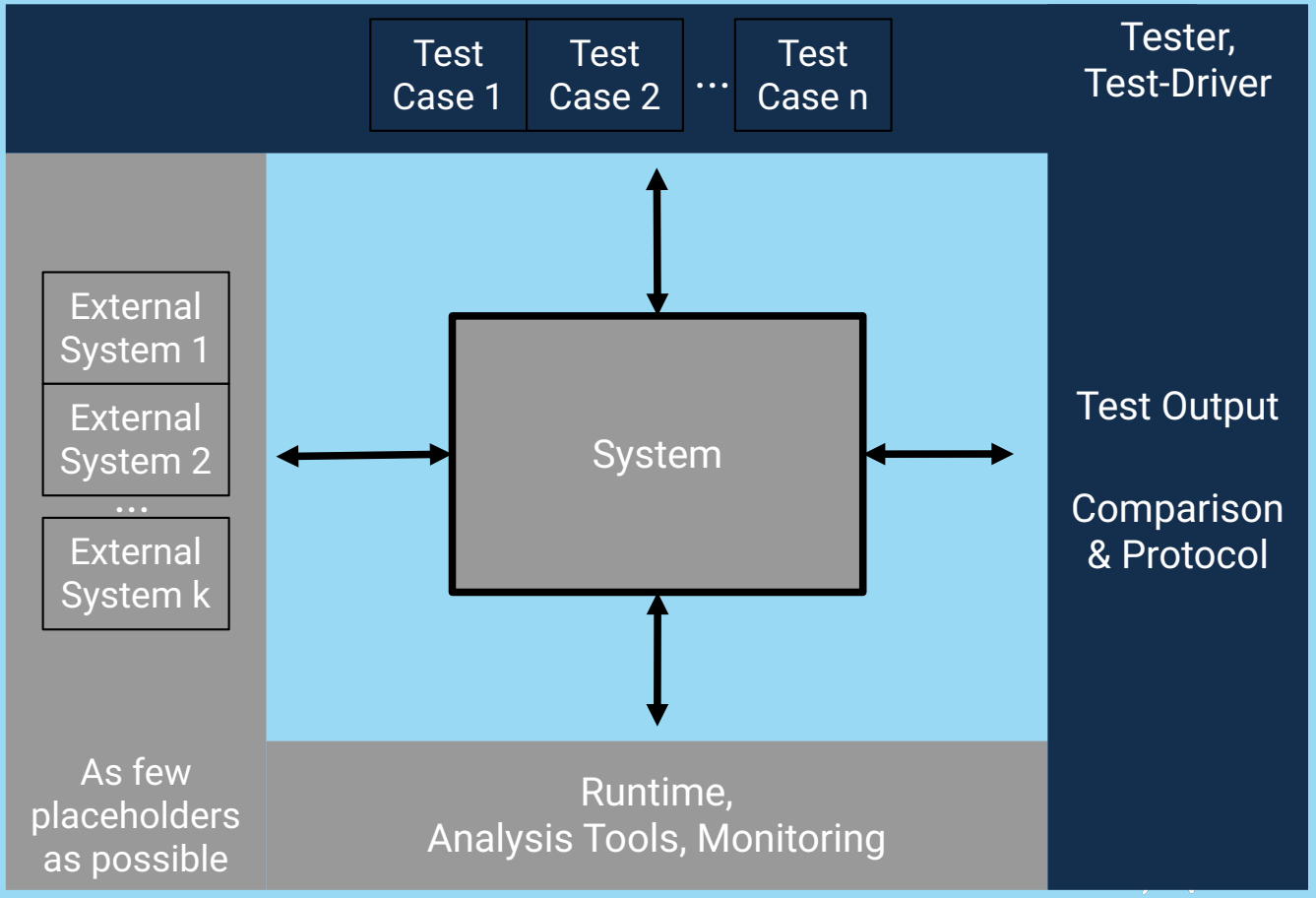


- Failure to correctly complete execution of functional tasks
  end-to-end tasks
- Failure of the system to work properly in the system environment(s)
  System environment(s)
- **System does not work as described in documentation**

- e.g. incorrect calculation results,
  incorrect / unexpected (non-)functional system behavior,
  incorrect control and/or data flows within the system

System Test – Test Environment

- Test environment should closely resemble later production environment
- Wherever possible: Use hardware or software that will actually be used



Question

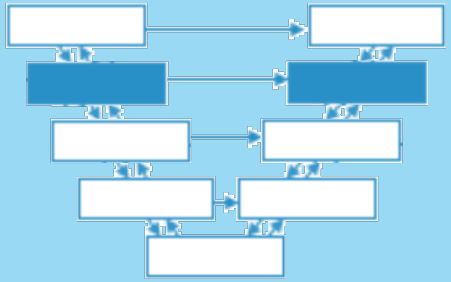
Should system tests be run in production?

Why? Why not?

Software maturing at the customer’s.

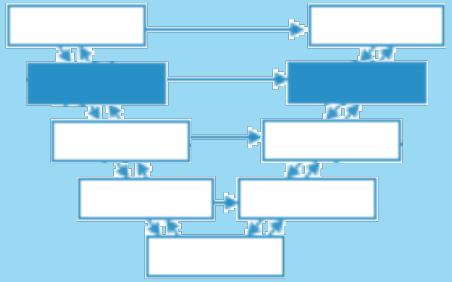
System Test – Test Environment

Disadvantages when testing in production



- Failures can affect the customer's productive environment
- Possible consequence: expensive system failures and data loss in customer system
- No / little control over configuration of the production environment; meaning potential changes to test conditions due to operation running parallel to the test
- System tests are difficult to reproduce / no longer reproducible

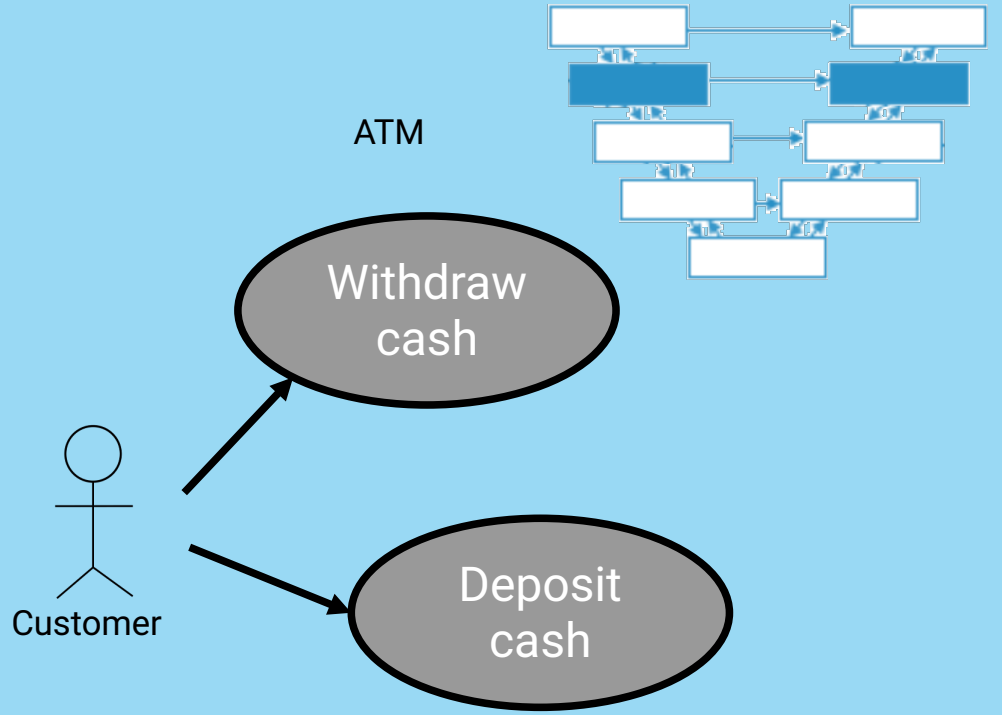
System Test – Test Strategy (1/2)



- Requirement-based testing
  - Test basis: Requirements document
  - At least one system test case derived and documented per requirement
  - Usually more than one test case to test a requirement
- Business process-based testing
  - **Create test scenarios that emulate typical business processes**
  - Business process analysis shows the **relevant business processes** (incl. context, people, companies, external systems, ...)
  - Priority based on frequency / relevance of business processes

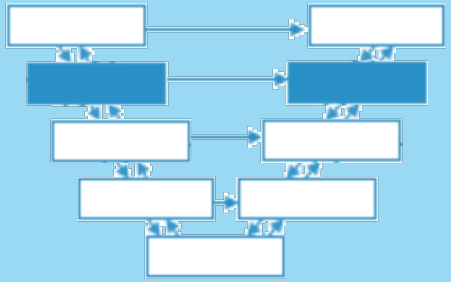
System Test – Test Strategy (2/2)

- Use case based testing
- System test cases represent typical system use
- User profile per user group with common usage pattern
- Derive test scenarios from usage patterns
- Priority of test scenarios based on frequency of actions monitored in operation



System Test – Test Strategy for NFRs (1/3)

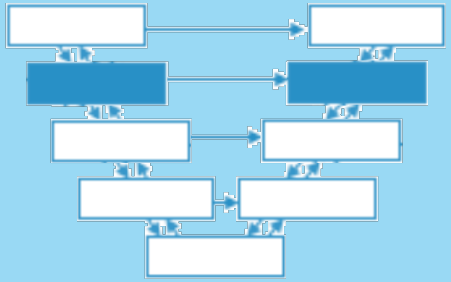
Non-functional requirements



- Performance test
  - Measurement of response time for use cases (with increasing load)
- Load test
  - Behavior of a system under changing load - usually between low load, typical load and peak load
- Volume/mass test
  - Check system behavior in relation to data volume (e.g. for very large files)

System Test – Test Strategy for NFRs (2/3)

Non-functional requirements

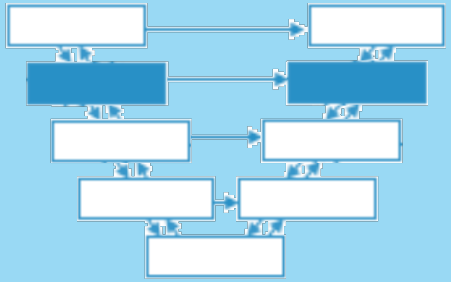


- Test of (data) security
  - against unauthorized system access or data access
- Reliability test
  - Continuous operation
    (failures per operating hour with usage profile x, ...)
- Resilience and Robustness test
  - against incorrect operation, incorrect programming, hardware failure
  - Testing of error handling and restart behavior
  - → ***Chaos Testing***

TH Aschaffenburg
university of applied sciences

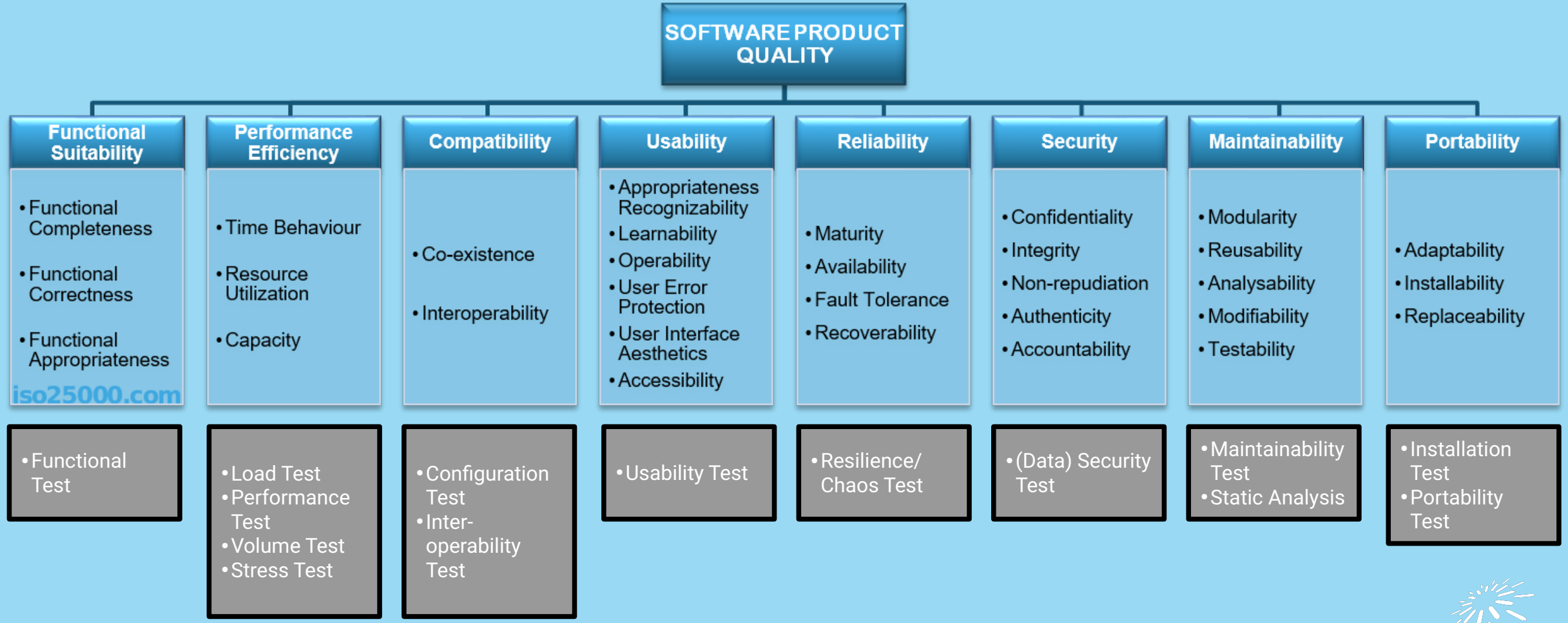
System Test – Test Strategy for NFRs (3/3)

Non-functional requirements

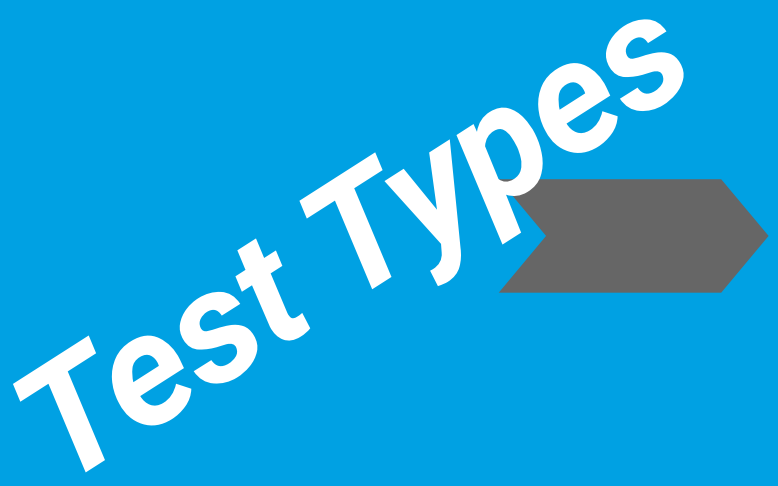


- Compatibility test / data conversion test
  - Check compatibility with existing systems
  - Import/export of databases
- Configuration test
  - Focus: different configurations of the system
- Usability test / usability test
  - Testing usability, comprehensibility of system output, ...
- Checking documentation
  - Compliance with system behavior (e.g. operating instructions)
- Check for changeability/maintainability
  - Comprehensibility of development documents, system structure, etc.

System Test – Quality Attributes according to ISO 25010



Chapter 2: Testing Throughout the Software Development Lifecycle



Testing in the Context of a Software Development Lifecycle

Test Levels and Test Types

Maintenance Testing

Learning Objectives for Testing Throughout the Software Development Lifecycle

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

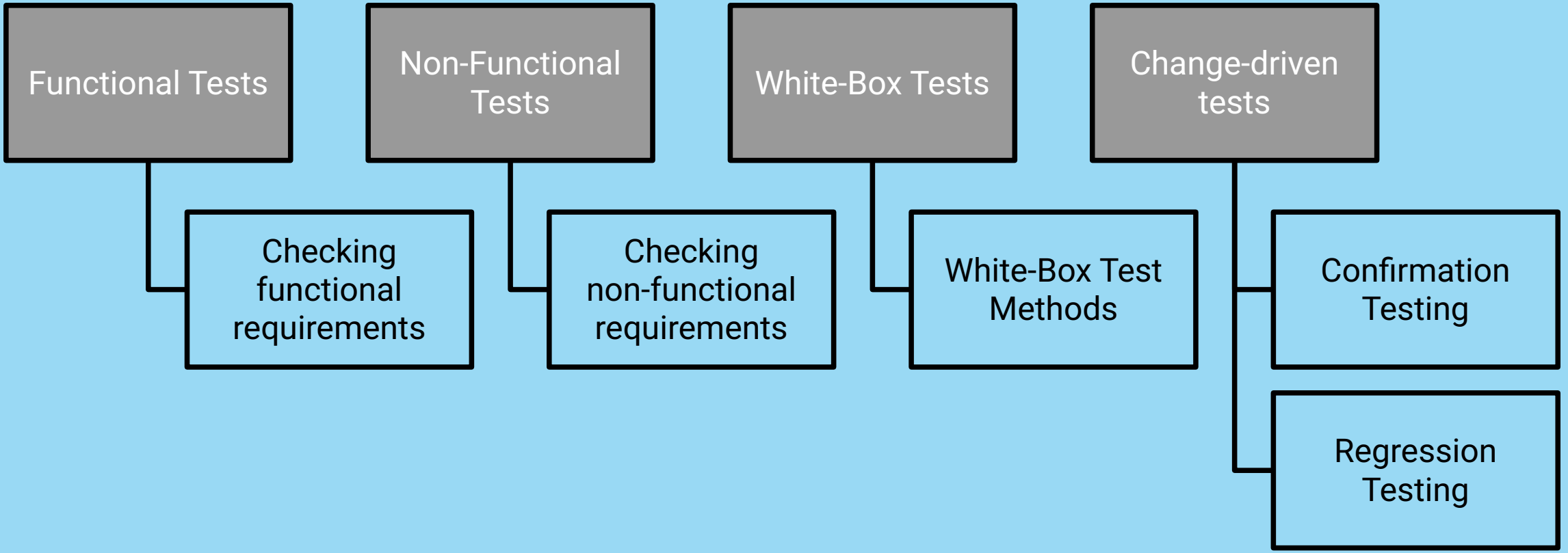
- **2.2 Test Levels and Test Types**
- FL-2.2.1 (K2) Distinguish the different test levels
- FL-2.2.2 (K2) Distinguish the different test types
- FL-2.2.3 (K2) Distinguish confirmation testing from regression testing
- **2.3 Maintenance Testing**
- FL-2.3.1 (K2) Summarize maintenance testing and its triggers

Test Types – Terms (1/2)

Test types group test activities according to test objectives:

- **Functional quality attributes**: complete, correct, appropriate?
- **Non-functional quality attributes**: reliable, performant, usable?
- Correctness and completeness of the **component** architecture
- Effects of **changes**

Test Types – Terms (2/2)



Test Levels and Test Types

- Each test type possible at each test level
  - Component test
  - Integration test
  - System test
  - Acceptance test
- Intensity of specific test types may vary per test level

Question

What is the difference between a Confirmation Test and a Regression Test?

Confirmation Test

- **Goal: Confirmation that defect has been removed**

- Defect fixed → perform tests that failed due to defect!
- New tests possible if defect was the absence of some functionality

Regression Test (1/3)

- Retesting of already tested software
- **Goal**: no new / previously masked defects in **unchanged areas**

- **Regression testing does NOT have the goal of finding errors!**

- Consider using regression tests even if software environment has changed
- Scope of the regression test?

Question

What is a good scope for regression tests?

Question

Impact Analysis

→ simple local changes may have unexpected effects

→ side effects on any (even distant) software possible

What is a good scope for regression tests?

Regression Test (2/3)

- Complete regression test
  - Repetition of all existing test cases
  - Same significance as test on initial version of the software
  - Also necessary if system environment has changed

- In practice: complete regression test almost always too expensive

Regression Test (3/3)

- Selection of regression test cases
  - Repeat high priority tests only
  - Omit special cases in functional tests
  - Focus on specific configurations (e.g. language, OS)
  - Focus on software impacted by changes

- Regression test suites often executed with few changes
- Excellent candidates for test automation

Perform
confirmation tests and regression tests
(repeatedly) at all test levels!

Chapter 2: Testing Throughout the Software Development Lifecycle

Testing in the Context of a Software Development Lifecycle

Test Levels and Test Types

Maintenance Testing

Learning Objectives for Testing Throughout the Software Development Lifecycle

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

- **2.2 Test Levels and Test Types**
- FL-2.2.1 (K2) Distinguish the different test levels
- FL-2.2.2 (K2) Distinguish the different test types
- FL-2.2.3 (K2) Distinguish confirmation testing from regression testing
- **2.3 Maintenance Testing**
- FL-2.3.1 (K2) Summarize maintenance testing and its triggers

TH Aschaffenburg
university of applied sciences

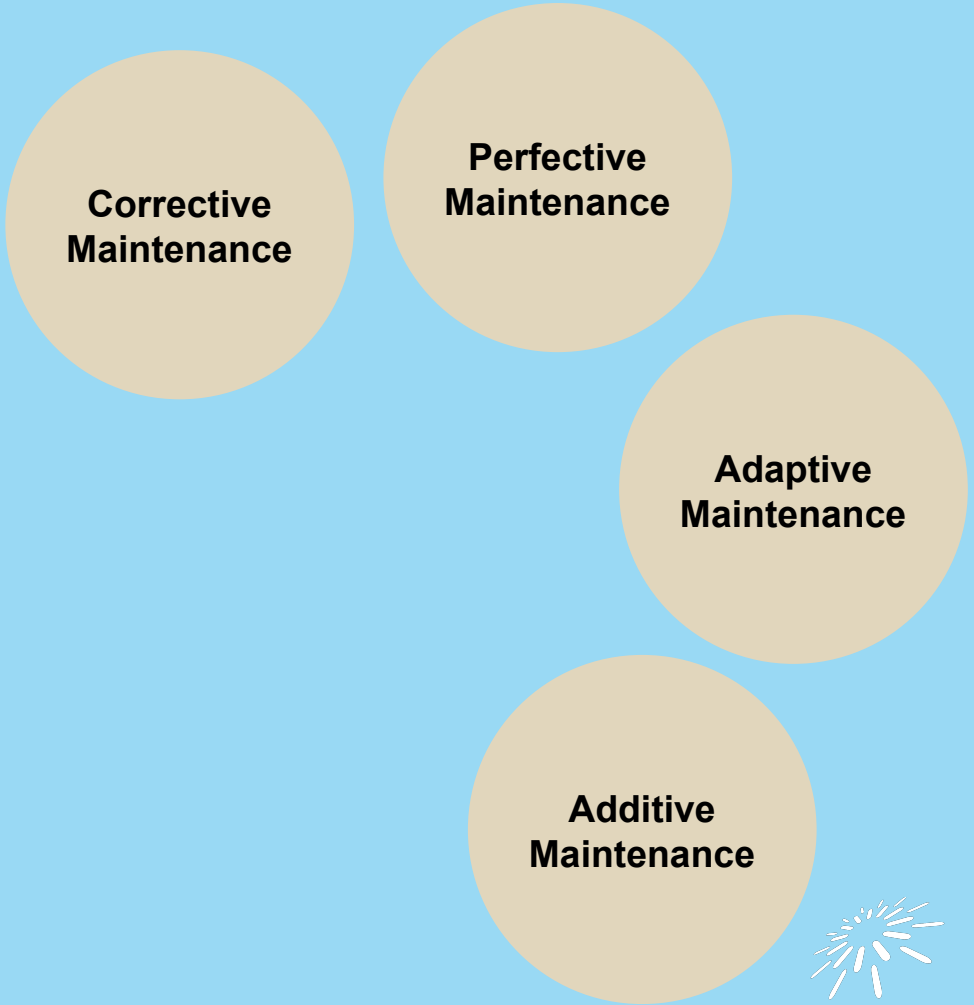
Testing a new Product Version (Maintenance Test)

- End of development after passing acceptance test and delivery?

- Reality:
  - Initial delivery is only the beginning of the life cycle
  - Often in use for years / decades after installation
  - Multiple changes to software, environment or configuration during this time
  - New version to be tested each time

Some Reasons for Maintenance – Which is which?

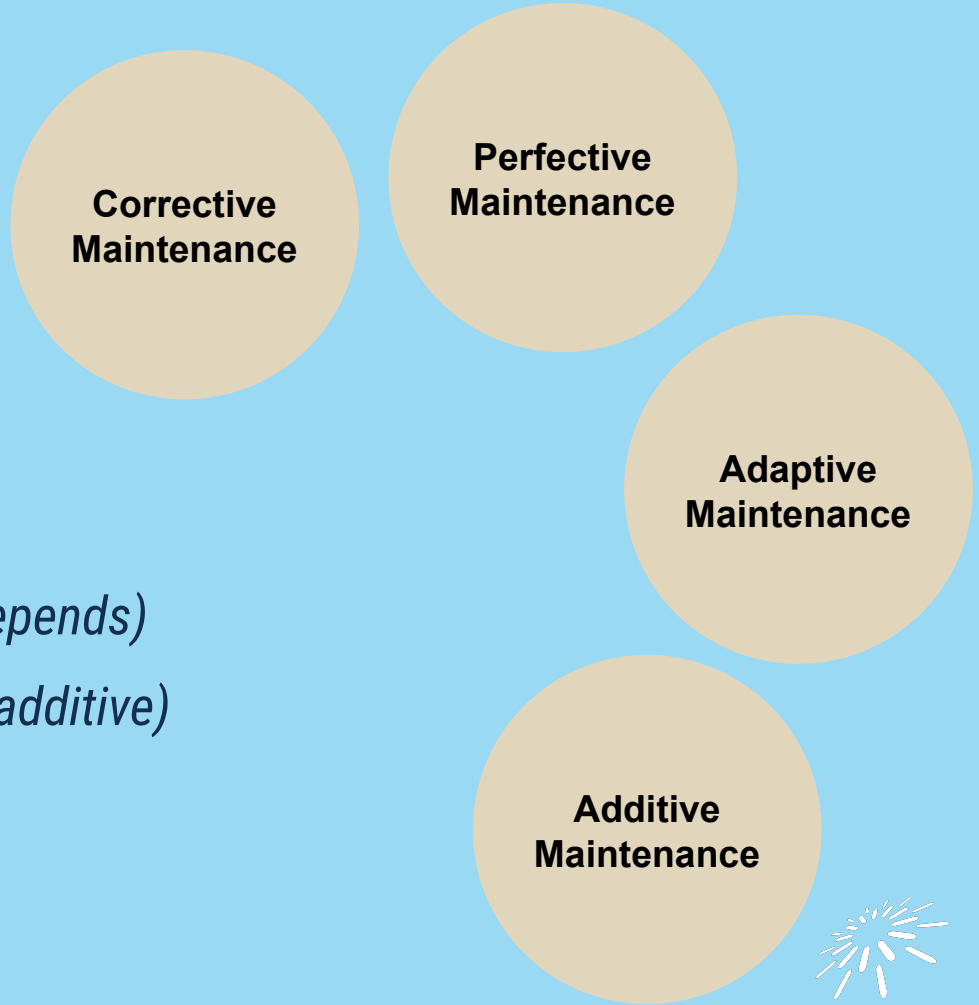
- Planned ongoing development
- Performance improvement
- Changing the environment
- Troubleshooting
- System refactoring
- Changes due to other changes in a neighboring system
- Implementation of a planned but unfinished functionality
- Extensions for a planned market expansion



TH Aschaffenburg
university of applied sciences

Some Reasons for Maintenance – Which is which?

- Planned ongoing development *(additive)*
- Performance improvement *(perfective)*
- Changing the environment *(adaptive)*
- Troubleshooting *(corrective)*
- System refactoring *(perfective)*
- Changes due to other changes in a neighboring system *(depends)*
- Implementation of a planned but unfinished functionality *(additive)*
- Extensions for a planned market expansion *(adaptive)*



TH Aschaffenburg
university of applied sciences

Reasons for Maintenance (1/2)

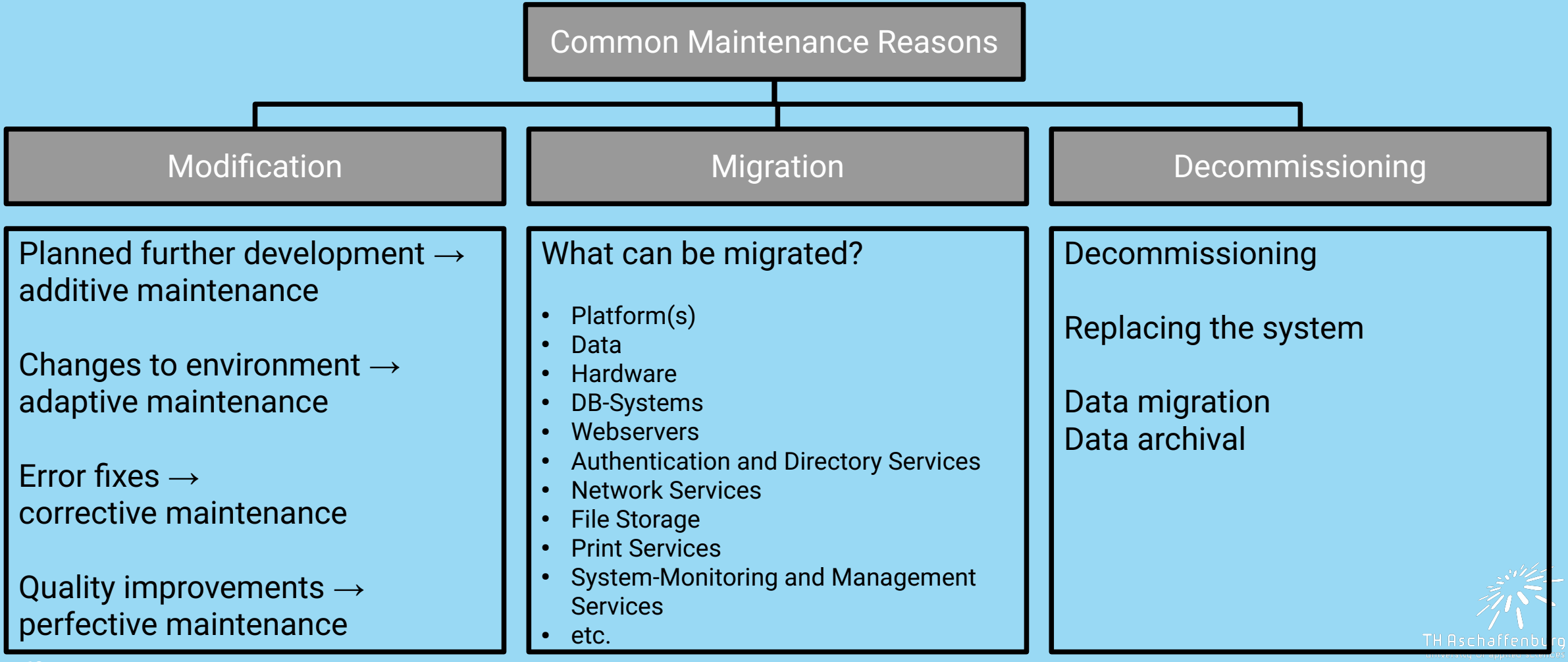
- Planned ongoing development *(additive maintenance)*
  - Modification and extension work planned from the outset
  - Normal product development
  - Software project not completed with delivery of the first version

- Troubleshooting *(corrective maintenance)*
  - Errors that occurred operation
  - Emergency fixes (“hot fixes”)

Reasons for Maintenance (2/2)

- Changing the environment *(adaptive maintenance)*
  - OS updates
  - DB management system updates
  - Upgrades of commercial software
  - Patches of external components
  - etc.
- Improving quality *(perfective maintenance)*
  - Improvement of quality factors, e.g. maintainability, performance, usability without changes in functional scope

Software Maintenance



Maintenance Test during Modification

- Maintenance release may require maintenance tests in several test stages
- Scope of maintenance tests depends on:
  - Change risk level (e.g. communication intensity)
  - Existing system size
  - Size of change
- Use impact analysis and traceability!

Maintenance Test during Migration/Decommissioning

- Maintenance test during migration
  - Includes tests during operation of new environment / changed software
- Maintenance test during decommissioning for ...
  - Data migration (to new system): Example migration of customer data in different formats
  - Archiving (for long retention periods)
  - Recovery procedure after archiving for long retention periods
  - Regression tests for functionality that remains in operation
- Conversion tests necessary
  - Test of data conversion software
  - Test of converted data

Take Away – Chapter 2 (1/2)

- General V-model
  - Test levels: Component test, integration test, system test (and acceptance test)
  - differentiates between verifying and validating tests
- **Component tests** test individual software modules
- **Integration tests** check the interaction of tested software modules
- Functional and non-functional **system tests** consider entire system
- **Acceptance test**: Client checks product for user acceptance and against acceptance criteria

Take Away – Chapter 2 (2/2)

- Basic test types
  - functional and non-functional test
  - structural and change-related testing
- Modification of software during life cycle
- **Testing of every changed version necessary!**
- Determine the scope of regression tests through impact analysis and risk assessment!

TH Aschaffenburg
university of applied sciences

Until next time!

Collaboration, Quality, Test

Prof. Dr. Jóakim von Kistowski

TH Aschaffenburg
university of applied sciences